Lab3__C401-E3

- **Team Name**: C401-E3
- **Repository (public)**: https://github.com/jot2003/Day-3-Lab-Chatbot-vs-react-agent
- **Submission Date**: 2026-04-06

A. Group Report

1. Executive Summary

Project goal: build a ReAct travel assistant for multi-step planning (weather + flights + budget), then compare it against a chatbot baseline.

- **Success Rate**
- Agent: 55/56 sessions reached valid `final_answer` (~98.2%).
- Chatbot: 4/4 sessions returned an answer (100%).
- **Key Outcome**
- Agent performs better on multi-step scenarios thanks to tool usage, structured traces, and citations.

2. System Architecture & Tooling

2.1 ReAct Loop

1. `Thought` - model reasons about next step.
2. `Action` - model calls exactly one tool.
3. `Observation` - tool returns structured JSON.
4. Repeat until `Final Answer` or `max_steps`.

2.2 Tool Inventory

Tool	Input	Use case
`get_weather`	`city`	Current weather + short forecast with resilient city query handling.

Tool	Input	Use case
`search_flights`	`origin, destination, departure_date`	One-way flight search (crawl-first with fallback).
`search_roundtrip_flights`	`origin, destination, departure_date, return_date`	Round-trip search from two one-way searches.
`search_itinerary_flights`	`segments_text`	Multi-leg itinerary search and aggregation.
`calculate_travel_budget`	budget/cost fields	Budget feasibility computation.

2.3 LLM Providers

- **Primary**: Gemini (`gemini-2.5-flash`)
- **Secondary**: provider factory supports OpenAI/local fallback paths.

3. Telemetry & Performance Dashboard

Source: `report/exports/*.csv` generated from `logs/`.

- **Total sessions**: 60 (56 agent, 4 chatbot)
- **Agent P50 latency**: ~15,316.5 ms/session
- **Observed max latency**: ~82,272 ms
- **Average prompt tokens**
- Agent: ~11,751.2
- Chatbot: ~314.2
- **Cost estimate**: available in `llm_metrics.csv` (`cost_estimate` field)

4. RCA - Failure Traces

4.1 Success trace (multi-step)

- **Input**: HAN -> DAD + weather + budget query
- **Trace**
- Action 1: `get_weather(Da Nang, VN)`
- Action 2: `search_flights(HAN, DAD, 2026-04-15)`
- Action 3: `calculate_travel_budget(...)`
- Final: budget feasibility conclusion using tool outputs

4.2 Failure trace (parser mismatch)

- **Input**: `get_weather(Da Nang, VN)` and `search_flights(origin='SGN', ...)`
- **Observation**
- weather used to fail with `expected 1 args, got 2`
- flight used to fail due to single-quote tokens
- **Root cause**
- comma splitting for one-arg tool
- inconsistent quote stripping
- **Fix**
- one-param tool keeps full string
- normalize both single and double quoted arguments
- regression tests added in `tests/test_travel_tools.py`

5. Ablation Studies & Experiments

5.1 Prompt v1 vs v2

- **Diff**: stricter tool-call guardrails, anti-hallucination instructions, better roundtrip/itinerary routing.
- **Result**: fewer invalid tool calls, improved stability on weather-only and multi-step prompts.

5.2 Chatbot vs Agent

Case	Chatbot	Agent	Winner
Simple weather	Fast, generic	Tool-backed with citation	Agent
Multi-step planning	Tends to miss evidence	Structured weather -> flight -> budget chain	Agent
Complex itinerary	Limited decomposition	Multi-leg tool path with aggregation	Agent

6. Flowchart & Group Insights

```
 User query
    |
    v
Thought + Action selection
    |
    v
Run one tool (weather / flight / budget)
    |
    v
Observation (JSON)
    |
```

```
+--> enough data? -- yes --> Final Answer
        |
        no
        v
   next Thought/Action loop
```

- **Insight 1**: user trust increases significantly with step trace + clickable evidence.
- **Insight 2**: parser/input normalization quality directly drives agent reliability.
- **Insight 3**: default one-way behavior avoids common intent mistakes.
- **Insight 4**: crawl/API fallback and transparent error reporting are mandatory for realtime travel use.

7. Production Readiness

- **Security**: keys in `.env`, not committed.
- **Guardrails**: `max_steps`, tool constraints, fallback policy.
- **Scalability**: modular split (`agent`, `tools`, `reporting`) ready for cache/queue/schema hardening.

B. Personal Reports

B.1 Hoàng Kim Trí Thành (2A202600372)

I. Technical Contribution

- Modules: `src/tools/weather.py`, `docs/OPENWEATHER_SETUP_VI.md`, `app.py`
- Highlights: weather query variants, direct weather evidence links, UI citation improvements.

II. Debugging Case Study

- Problem: model hallucinated weather API limitations.
- Log source: `logs/2026-04-06.log` (`AGENT_LLM_STEP`, `AGENT_OBSERVATION`).
- Diagnosis: prompt constraints were not strict enough.
- Fix: enforce observation-grounded explanations in system prompt.

III. Personal Insights

1. Thought blocks improve decomposition quality vs single-shot chatbot.
2. Agent can degrade when external tools/API become unstable.
3. Observation feedback is critical for corrective next actions.

IV. Future Improvements

- Add weather cache by city/time window.
- Standardize error-language policy for user-facing responses.
- Reduce token usage in weather-only scenarios.

B.2 Đặng Đình Tú Anh (2A202600019)

I. Technical Contribution

- Modules: `src/tools/flights.py`, `src/tools/registry.py`, `app.py`
- Highlights: crawl-first flight pipeline, roundtrip/itinerary tools, parser hardening, citation improvements.

II. Debugging Case Study

- Problem: 422 errors when arguments contained single quotes.
- Log source: `logs/2026-04-06.log` flight observations.
- Diagnosis: quoted token normalization was incomplete.
- Fix: parser now strips both single/double quotes + regression tests.

III. Personal Insights

1. ReAct provides more reliable chain-of-evidence than single-shot responses.
2. Realtime flight data quality depends on source and timing variance.
3. Observation-driven retries improve final answer quality.

IV. Future Improvements

- Async workers for flight calls and route-level caching.
- Add stricter pre-call parameter validation layer.
- Further reduce unnecessary loops on straightforward queries.

B.3 Quách Gia Được (2A202600423)

I. Technical Contribution

- Modules: `src/agent/agent.py`, `main.py`, `app.py`
- Highlights: iter-based streaming execution, loop control, consistent behavior between CLI and Streamlit.

II. Debugging Case Study

- Problem: occasional wrong tool selection for weather-only prompts.
- Diagnosis: intent-to-tool guidance in prompt was under-specified.
- Fix: stronger tool-routing instructions and examples.

III. Personal Insights

1. ReAct makes model reasoning inspectable.
2. More steps can increase accumulated failure risk.
3. Observation blocks reduce random action selection.

IV. Future Improvements

- Planner/executor split for complex orchestration.
- Pre-execution policy checks on generated actions.
- Early-stop heuristics when sufficient evidence is already present.

B.4 Phạm Quốc Dũng (2A202600490)

I. Technical Contribution

- Modules: `tests/test_travel_tools.py`, `src/tools/demo_fallback.py`, `.env.example`
- Highlights: expanded tests for parser/date/roundtrip/itinerary and normalized mock outputs.

II. Debugging Case Study

- Problem: date assertions broke due to source format differences.
- Diagnosis: tests relied on unstable raw datetime formats.
- Fix: move assertions to normalized date fields.

III. Personal Insights

1. ReAct supports iterative correction from observations.
2. Chatbot can be more stable in trivial single-step Q&A.
3. Test coverage is essential for tool-driven systems.

IV. Future Improvements

- Automated provider matrix testing.
- More adversarial input-format tests.
- CI benchmark tracking for latency/token trends.

B.5 Nguyễn Thành Nam (2A202600205)

I. Technical Contribution

- Modules: `src/reporting/log_summary.py`, `scripts/summarize_logs.py`, `report/exports/*`
- Highlights: telemetry aggregation and CSV exports for evaluation.

II. Debugging Case Study

- Problem: dashboard metrics were inconsistent across runs.
- Diagnosis: mixed demo and realtime sessions in the same analysis windows.
- Fix: clearer filtering/labeling and sampling rules during summary.

III. Personal Insights

1. Agent traces improve explainability for grading and debugging.
2. External dependency failures are central reliability risks.
3. Structured logs convert failures into actionable engineering tasks.

IV. Future Improvements

- Batch telemetry pipeline and long-term metrics storage.
- Redaction for sensitive fields before sharing logs.
- Automated alerting on latency/error thresholds.

C. Submission Notes

- This repository is public and used as the single team submission codebase.
- Report file naming follows the required format: `Lab3__C401-E3`.
- If the instructor requires one combined report artifact, this file is the final integrated version (group + personal).